

# Wheeled Biped Robot (RoTino)

Manuale Tecnico Completo:  
Architettura del Codice e Fondamenti Teorici  
Analisi file per file con collegamento alla teoria del controllo

Analisi Tecnica Approfondita

25 settembre 2026

# Indice

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione e Architettura del Progetto</b>                         | <b>2</b>  |
| 1.1      | Obiettivo del Documento . . . . .                                       | 2         |
| 1.2      | Albero Completo dei File . . . . .                                      | 2         |
| <b>2</b> | <b>rotino_description: Il Modello Dinamico del Robot</b>                | <b>4</b>  |
| 2.1      | Ruolo del Pacchetto . . . . .                                           | 4         |
| 2.2      | File: kinematics.py . . . . .                                           | 4         |
| 2.2.1    | Ruolo . . . . .                                                         | 4         |
| 2.2.2    | Riga 9–17: Conversione RPY a matrice di rotazione . . . . .             | 4         |
| 2.2.3    | Riga 20–24: Formula di Rodrigues (Asse-Angolo) . . . . .                | 5         |
| 2.2.4    | Riga 54–71: Cinematica Diretta ad Albero . . . . .                      | 5         |
| 2.3      | File: model.py . . . . .                                                | 6         |
| 2.3.1    | Ruolo . . . . .                                                         | 6         |
| 2.3.2    | Riga 28–48: Struttura dati WBRParams (Tabella 1 del Paper) . . . . .    | 6         |
| 2.3.3    | Riga 151–178: Centroidi Equivalente (Eqs. 2-3 del Paper) . . . . .      | 6         |
| 2.3.4    | Riga 180–195: Coefficienti VL-WIP (Eq. 14 del Paper) . . . . .          | 7         |
| 2.3.5    | Riga 211–223: FK Analitica 2D delle Gambe e Jacobiano . . . . .         | 7         |
| 2.4      | File: planar_trajectory.py . . . . .                                    | 8         |
| 2.4.1    | Ruolo . . . . .                                                         | 8         |
| 2.4.2    | Riga 50–63: Legge Oraria Quintica e Curvatura . . . . .                 | 8         |
| <b>3</b> | <b>rotino_smc: Sliding Mode Control</b>                                 | <b>10</b> |
| 3.1      | Architettura del Controllore . . . . .                                  | 10        |
| 3.2      | File: estimator.py . . . . .                                            | 10        |
| 3.2.1    | Ruolo . . . . .                                                         | 10        |
| 3.2.2    | Riga 26–37: Predizione e Correzione del Filtro di Kalman . . . . .      | 10        |
| 3.3      | File: controller.py (SMC) . . . . .                                     | 11        |
| 3.3.1    | Riga 576–587: Pitch Super-Twisting (Algoritmo di Levant) . . . . .      | 11        |
| 3.3.2    | Righe di Aggiornamento dell'Integratore W nel Loop Principale . . . . . | 11        |
| 3.3.3    | Riga 555–574: Task-Space VMC (Virtual Model Control) . . . . .          | 12        |
| 3.3.4    | Loop Esterno di Velocita' (Non-Minimum Phase) . . . . .                 | 12        |
| <b>4</b> | <b>rotino_mpc: Model Predictive Control</b>                             | <b>14</b> |
| 4.1      | Architettura MPC . . . . .                                              | 14        |
| 4.2      | File: solvers.py . . . . .                                              | 14        |
| 4.2.1    | Riga 15–30: CARE e Guadagno LQR Ottimale . . . . .                      | 14        |
| 4.2.2    | Riga 33–45: LQRSchedule (TV-LQR su griglia di lunghezze) . . . . .      | 14        |
| 4.2.3    | Riga 48–58: Fast QP con Algoritmo FISTA . . . . .                       | 15        |
| 4.2.4    | Riga 61–76: Condensazione del Problema MPC . . . . .                    | 15        |

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>5</b> | <b>rotino_pid: Controllore Baseline PD</b>             | <b>17</b> |
| 5.1      | File: controller.py (PID)                              | 17        |
| 5.1.1    | Legge PD di Bilanciamento                              | 17        |
| 5.1.2    | Servo PD di Posizione per le Gambe                     | 17        |
| <b>6</b> | <b>rotino_benchmark: Infrastruttura di Validazione</b> | <b>18</b> |
| 6.1      | File: campaign.py                                      | 18        |
| 6.2      | File: logger.py                                        | 18        |
| 6.3      | File: plot.py – I 9 Pannelli di Analisi                | 19        |
| <b>7</b> | <b>Conclusioni</b>                                     | <b>20</b> |

# Capitolo 1

## Introduzione e Architettura del Progetto

### 1.1 Obiettivo del Documento

Questo manuale ha lo scopo di collegare in modo esplicito e rigoroso ogni singolo file sorgente del progetto RoTino alla teoria del controllo e della robotica da cui trae origine. Per ciascun file viene presentata prima una descrizione del ruolo architetturale, poi le righe di codice piu' significative con la corrispondente derivazione matematica.

### 1.2 Albero Completo dei File

Di seguito e' riportata la struttura completa del workspace ROS 2 Humble:

```
rotino_ws/
|-- src/
|   |-- rotino_description/
|   |   |-- rotino_description/
|   |   |   |-- model.py          <-- Parametri VL-WIP dall'URDF
|   |   |   |-- kinematics.py     <-- FK analitica e CoM globale
|   |   |   '-- planar_trajectory.py <-- Legge oraria S-curve quintica
|   |   '-- urdf/
|   |       '-- rotino.urdf.xacro  <-- Geometria, masse, giunti
|   |
|   |-- rotino_pid/
|   |   '-- rotino_pid/
|   |       '-- controller.py      <-- Bilanciamento PD + jump FSM
|   |
|   |-- rotino_smc/
|   |   '-- rotino_smc/
|   |       |-- controller.py      <-- Super-Twisting + Task-Space VMC
|   |       '-- estimator.py       <-- Filtro di Kalman lineare
|   |
|   |-- rotino_mpc/
|   |   '-- rotino_mpc/
|   |       |-- controller.py      <-- Nodo ROS 2, loop di controllo
|   |       '-- solvers.py         <-- QP FISTA, LQR, condensazione
|   |
|   '-- rotino_benchmark/
```

```
|      '-- rotino_benchmark/
|      |-- campaign.py      <-- Orchestrazione run sequenziali
|      |-- logger.py        <-- Logging sincrono ad alta freq.
|      |-- plot.py          <-- 9 pannelli di analisi
|      '-- compare.py       <-- Confronto tra run
'-- docs/
    '-- Architettura_e_Codice.tex  <- Questo documento
```

## Capitolo 2

# rotino\_description: Il Modello Dinamico del Robot

### 2.1 Ruolo del Pacchetto

Questo pacchetto ha due compiti fondamentali: (1) definire la geometria e le inerzie del robot tramite URDF/Xacro, e (2) fornire a tutti gli altri pacchetti una libreria Python che estrae automaticamente i parametri dinamici dal modello URDF, evitando valori scritti a mano che potrebbero discostarsi dalla simulazione.

### 2.2 File: kinematics.py

#### 2.2.1 Ruolo

Implementa la **cinematica diretta (Forward Kinematics)** del robot traversando l'albero URDF a partire dalla radice, senza dipendenze da librerie esterne come Pinocchio o MoveIt. Calcola il centro di massa globale tramite media ponderata delle masse.

#### 2.2.2 Riga 9–17: Conversione RPY a matrice di rotazione

**Teoria:** Una rototraslazione rigida nello spazio 3D si rappresenta con la matrice di rotazione  $\mathbf{R} \in SO(3)$ . Per la convenzione RPY (Roll, Pitch, Yaw) usata da URDF e ROS, la rotazione composta e' il prodotto:  $\mathbf{R} = R_z(\psi)R_y(\phi)R_x(\theta)$ , la cui forma esplicita e' la classica matrice di rotazione di Eulero ZYX.

```
1 def rpy_to_matrix(r, p, y):
2     cr, sr = np.cos(r), np.sin(r)
3     cp, sp = np.cos(p), np.sin(p)
4     cy, sy = np.cos(y), np.sin(y)
5     # Prodotto R_z * R_y * R_x in forma esplicita (convenzione ZYX)
6     return np.array([
7         [cy*cp, cy*sp*sr - sy*cr, cy*sp*cr + sy*sr],
8         [sy*cp, sy*sp*sr + cy*cr, sy*sp*cr - cy*sr],
9         [-sp,      cp*sr,      cp*cr],
10    ])
```

Listing 2.1: kinematics.py rr. 9-17: Matrice di rotazione da angoli RPY

Questa funzione viene richiamata ogni volta che si propaga la trasformazione di un giunto fisso dell'URDF, che specifica la sua posa relativa tramite attributo `rpy` nella tag `<origin>`.

### 2.2.3 Riga 20–24: Formula di Rodrigues (Asse-Angolo)

**Teoria (Formula di Rodrigues):** Data una rotazione di angolo  $\alpha$  attorno all'asse unitario  $\hat{\mathbf{k}}$ , la matrice di rotazione è:

$$\mathbf{R} = \mathbf{I} + \sin \alpha [\hat{\mathbf{k}}]_{\times} + (1 - \cos \alpha) [\hat{\mathbf{k}}]_{\times}^2$$

dove  $[\cdot]_{\times}$  è la matrice antisimmetrica (skew-symmetric) tale che  $[\mathbf{k}]_{\times} \mathbf{v} = \mathbf{k} \times \mathbf{v}$ .

```
1 def axis_angle_to_matrix(axis, angle):
2     k = np.asarray(axis, dtype=float)
3     k = k / np.linalg.norm(k)
4     # kx e' la matrice skew-symmetric [k]_x
5     kx = np.array([[0.0, -k[2], k[1]],
6                    [k[2], 0.0, -k[0]],
7                    [-k[1], k[0], 0.0]])
8     # Formula di Rodrigues esatta
9     return np.eye(3) + np.sin(angle) * kx + (1.0 - np.cos(angle)) * (kx
10 @ kx)
```

Listing 2.2: kinematics.py rr. 20-24: Formula di Rodrigues

Questa è la formula canonica per costruire la matrice di rotazione  $R \in SO(3)$  attorno a un asse arbitrario. Viene usata in `link_frames` per ogni giunto revoluto, dove l'asse è quello specificato dall'URDF e l'angolo è la variabile di giunto  $q_i$ .

### 2.2.4 Riga 54–71: Cinematica Diretta ad Albero

**Teoria (FK per strutture ad albero):** La cinematica diretta propaga la trasformazione omogenea  ${}^0T_n = {}^0T_1 \cdot {}^1T_2 \cdots {}^{n-1}T_n$  dal nodo radice verso le foglie. In forma compatta, per ogni link figlio:  $p_{child} = p_{parent} + R_{parent} \cdot p_{joint}$ ,  $R_{child} = R_{parent} \cdot R_{joint} \cdot R_q(q_i)$ .

```
1 def link_frames(self, base_pos, base_rot, joint_positions):
2     frames = {}
3     stack = [(self.root, base_rot, base_pos)]
4     while stack:
5         name, rot, pos = stack.pop()
6         frames[name] = (rot, pos)
7         for joint in self.child_joints.get(name, []):
8             origin = joint.origin
9             xyz = np.array(origin.xyz if origin.xyz else [0., 0., 0.])
10            rpy = origin.rpy if origin.rpy else [0., 0., 0.]
11            # p_child = p_parent + R_parent * p_joint_origin
12            child_pos = pos + rot @ xyz
13            # R_child = R_parent * R_joint_origin
14            child_rot = rot @ rpy_to_matrix(*rpy)
15            if joint.type in ('revolute', 'continuous'):
16                # Applica la rotazione del giunto con la formula di
17                # Rodrigues
18                child_rot = child_rot @ axis_angle_to_matrix(
19                    joint.axis, joint_positions.get(joint.name, 0.0))
20            stack.append((joint.child, child_rot, child_pos))
21    return frames
```

Listing 2.3: kinematics.py rr. 54-71: FK iterativa via stack DFS

L'approccio DFS (Depth First Search) tramite stack esplicito è equivalente alla ricorsione ma evita l'overhead dello stack Python per alberi profondi.

## 2.3 File: model.py

### 2.3.1 Ruolo

Questo e' il nucleo analitico del progetto. Implementa il modello **VL-WIP (Variable-Length Wheeled Inverted Pendulum)** di Cui et al., *Micromachines* 2022, 13, 747, calcolando automaticamente tutti i parametri dinamici dalla geometria URDF. Ogni calcolo ha un riferimento esplicito all'equazione del paper.

### 2.3.2 Riga 28–48: Struttura dati WBRParams (Tabella 1 del Paper)

```
1 @dataclass
2 class WBRParams:
3     m_w: float # massa di una ruota [kg] -> Tab.1: m_w
4     I_w: float # inerzia di spin della ruota [kg m2] -> Tab.1: I_w
5     r: float # raggio della ruota [m] -> Tab.1: r
6     d: float # carreggiata [m] -> Tab.1: d
7     m_b: float # massa corpo superiore [kg] -> Tab.1: m_b
8     m_1: float # massa dello stinco [kg] -> Tab.1: m_1
9     m_2: float # massa della coscia [kg] -> Tab.1: m_2
10    l_1: float # lunghezza stinco: ginocchio->ruota -> Tab.1: l_1
11    l_2: float # lunghezza coscia: anca->ginocchio -> Tab.1: l_2
12    L_max: float # lunghezza massima gamba = l1 + l2 -> Tab.1: L
13    mu: float # coeff. di attrito ruota/suolo -> Tab.1: mu
14    leg_damping: float # smorzamento viscoso giunto [Nm s/rad]
```

Listing 2.4: model.py rr. 28-48: Struttura parametri VL-WIP

Questi parametri corrispondono esattamente alla Tabella 1 del paper di riferimento. L'estrazione automatica garantisce coerenza tra simulazione e modello analitico.

### 2.3.3 Riga 151–178: Centroide Equivalente (Eqs. 2-3 del Paper)

**Teoria (Eqs. 2-3):** Il WBR viene modellato come un pendolo inverso a lunghezza variabile. Il corpo superiore (gambe + torso) e' sostituito da un punto materiale equivalente di massa  $m_b$ :

$$S_C = \frac{\sum_i m_i x_i}{m_b}, \quad Z_C = \frac{\sum_i m_i z_i}{m_b}, \quad l = \sqrt{S_C^2 + Z_C^2}, \quad \theta = \arctan\left(\frac{S_C}{Z_C}\right)$$

dove la somma si estende a tutti i link tranne le ruote, e le coordinate sono nel frame dell'asse ruote.

```
1 def equivalent_centroid(self, hip, knee, pitch=0.0):
2     R = rpy_to_matrix(0.0, pitch, 0.0)
3     frames = self.kin.link_frames(np.zeros(3), R, self.leg_joints(hip,
4     knee))
5     # Punto medio dell'asse ruote
6     axle = 0.5*(frames[LEFT_WHEEL_LINK][1] + frames[RIGHT_WHEEL_LINK
7     ] [1])
8     # CoM ponderato del corpo superiore (tutto tranne le ruote)
9     masses = np.array([_link(self.robot, n).inertial.mass for n in names
10    ])
11    coms = np.array([self._link_com(frames, n) for n in names])
12    com = masses @ coms / masses.sum()
13    # Eq. (2)-(3): distanza S_C, Z_C rispetto all'asse ruote
14    S_C = float(com[0] - axle[0])
15    Z_C = float(com[2] - axle[2])
```

```

13     return {
14         'l':      math.hypot(S_C, Z_C),      # lunghezza del pendolo
equivalente
15         'theta': math.atan2(S_C, Z_C),      # angolo rispetto alla
verticale
16         'I_y':    float(I[1, 1]),           # inerzia di pitch del
corpo sup.
17         'z_b':    float(-axle[2]),          # altezza asse ruote da
terra
18     }

```

Listing 2.5: model.py rr. 151-178: Centroide equivalente (Eqs. 2-3)

### 2.3.4 Riga 180–195: Coefficienti VL-WIP (Eq. 14 del Paper)

**Teoria (Eq. 14):** Linearizzando le equazioni di Lagrange del WBR attorno all'equilibrio (robot verticale, ruote che rotolano senza slittamento), si ottiene il modello lineare:

$$\ddot{s} = a_1\theta + b_1(\tau_L + \tau_R), \quad \ddot{\theta} = a_2\theta + b_2(\tau_L + \tau_R)$$

dove  $a_2 > 0$  implica l'instabilità del pendolo (autovalore positivo). I coefficienti dipendono dalla lunghezza  $l$  del pendolo, variabile con la postura.

```

1 def vlwip_coefficients(self, l, I_y=None, I_z=None):
2     p = self.p
3     if I_y is None:
4         I_y = p.m_b * l * l / 3.0 # approx. del paper: corpo a bastone
5     mb, mw, Iw, r, d = p.m_b, p.m_w, p.I_w, p.r, p.d
6     # Denominatore comune (determinante della matrice di inerzia)
7     den = (2*Iw*(I_y + mb*l*l)
8           + (2*l*l*mb*mw + I_y*(mb + 2*mw))*r*r)
9     return {
10         'a1': -G*l*l*mb*mb*r*r / den,      # accoppiamento s <- theta
11         'a2':  G*l*mb*(2*Iw+(mb+2*mw)*r*r)/den, # instabilita' pendolo
12         'b1':  r*(I_y + l*mb*(l+r)) / den,   # guadagno coppia -> s
13         'b2': -(2*Iw + r*(l*mb+(mb+2*mw)*r))/den, # guadagno coppia ->
theta
14         'b3':  d*r / (2*I_z*r*r + d*d*(Iw+mw*r*r)), # guadagno yaw
15     }

```

Listing 2.6: model.py rr. 180-195: Coefficienti VL-WIP (Eq. 14)

Il fatto che  $b_2 < 0$  è fondamentale: nella funzione `_pitch_super_twisting`, la coppia fisica alle ruote si calcola dividendo per  $b_2$ , il che inverte automaticamente il segno del segnale di controllo.

### 2.3.5 Riga 211–223: FK Analitica 2D delle Gambe e Jacobiano

**Teoria (Catena cinematica planare):** Per un robot a 2 link planari (anca-ginocchio-ruota), la posizione del centro ruota nel frame base è:  $\mathbf{p}_{wheel} = \mathbf{p}_{hip} + R(\phi_{hip})\mathbf{p}_{knee|hip} + R(\phi_{hip} + \phi_{knee})\mathbf{p}_{wheel|knee}$ . Il Jacobiano  $\mathbf{J} = \partial\mathbf{p}_{wheel}/\partial(\phi_{hip}, \phi_{knee})$  si calcola derivando analiticamente rispetto agli angoli di giunto.

```

1 def leg_fk(self, hip, knee):
2     kx, kz = self.knee_xz # offset ginocchio nel frame dell'anca [m]
3     wx, wz = self.wheel_xz # offset ruota nel frame del ginocchio [m]

```

```

4
5     def rot(q, x, z):
6         c, s = math.cos(q), math.sin(q)
7         # Punto ruotato e sua derivata rispetto all'angolo q
8         return (np.array([x*c + z*s, -x*s + z*c]), # posizione
9                 np.array([-x*s + z*c, -x*c - z*s])) # dp/dq (colonna di
10
11     J)
12
13     pk, dpk = rot(hip, kx, kz) # posizione ginocchio e sua
14     derivata
15     pw, dpw = rot(hip + knee, wx, wz) # posizione ruota e sua
16     derivata
17     # J[:,0] = d(p_ruota)/d(hip): ruotare l'anca muove sia ginocchio che
18     ruota
19     # J[:,1] = d(p_ruota)/d(knee): ruotare il ginocchio muove solo la
20     ruota
21     J = np.column_stack((dpk + dpw, dpw))
22     return self.hip_xz + pk + pw, J

```

Listing 2.7: model.py rr. 211-223: FK 2D e Jacobiano analitico

## 2.4 File: planar\_trajectory.py

### 2.4.1 Ruolo

Genera la traiettoria planare (S-curve laterale) che il robot segue durante la navigazione in avanzamento e sterzata, con una legge oraria quintica che garantisce accelerazione zero agli estremi (jerk minimizzato).

### 2.4.2 Riga 50–63: Legge Oraria Quintica e Curvatura

**Teoria (Polinomio quintico per profili smooth):** Un profilo di posizione  $s(t)$  con  $s(0) = \dot{s}(0) = \ddot{s}(0) = 0$  e  $s(T) = L$ ,  $\dot{s}(T) = \ddot{s}(T) = 0$  (“profilo a S”) si ottiene con il polinomio quintico normalizzato  $\tau = t/T$ :

$$s(\tau) = L(10\tau^3 - 15\tau^4 + 6\tau^5)$$

La velocità  $\dot{s} = L(30\tau^2 - 60\tau^3 + 30\tau^4)/T$  ha profilo trapezoidale arrotondato. La curvatura della traiettoria planare è  $\kappa = y''/(1 + y'^2)^{3/2}$  e il yaw\_rate desiderato è  $\dot{\psi} = \kappa \cdot v$ .

```

1 def _time_law(self, t):
2     tau = min(max(t / self.duration, 0.0), 1.0) # normalizzazione
3     [0,1]
4     s = self.length * (10*tau**3 - 15*tau**4 + 6*tau**5) # posizione
5     v = self.length * (30*tau**2 - 60*tau**3 + 30*tau**4) / self.
6     duration # velocità
7     return s, v
8
9 def sample(self, t):
10     s, v = self._time_law(t)
11     # Inversione della curva lunghezza-arco (tabella precalcolata)
12     x = float(np.interp(s, self.s_table, self.x_table))
13     dy = self._dy(x) # derivata della S-curve
14     spaziale
15     heading = math.atan(dy) # angolo tangente alla
16     curva

```

```

13     # Curvatura della curva planare: kappa = y'' / (1 + y'^2)^(3/2)
14     curvature = self._ddy(x) / (1.0 + dy*dy)**1.5
15     # yaw_rate desiderato = curvatura * velocita' lineare
16     return x, float(self._y(x)), heading, v, curvature * v

```

Listing 2.8: planar\_trajectory.py rr. 50-63: Legge quintica e campionamento

## Capitolo 3

# rotino\_smc: Sliding Mode Control

### 3.1 Architettura del Controllore

Il pacchetto implementa tre leggi di controllo simultanee: (1) pitch Super-Twisting sulle ruote, (2) leg Task-Space SMC (VMC esteso), (3) yaw PD+I con feedforward di attrito.

### 3.2 File: estimator.py

#### 3.2.1 Ruolo

Implementa il **Filtro di Kalman lineare a due stati**  $[p, v]$  per stimare posizione e velocità del corpo superiore, fondendo le misure dell'accelerometro IMU (aggiornamento veloce) con l'odometria delle ruote combinata alla cinematica delle gambe (correzione lenta e accurata).

#### 3.2.2 Riga 26–37: Predizione e Correzione del Filtro di Kalman

**Teoria (Kalman Discreto):** Per il sistema  $\mathbf{x}_{k+1} = F\mathbf{x}_k + Bu_k + w_k$  con  $w_k \sim \mathcal{N}(0, Q)$  e osservazione  $y_k = \mathbf{x}_k + v_k$  con  $v_k \sim \mathcal{N}(0, R)$ , le equazioni ottimali sono:

Predizione:  $\hat{x}^- = F\hat{x}_{k-1} + Bu_{k-1}$ ,  $P^- = FP_{k-1}F^T + Q$

Correzione:  $K = P^-(P^- + R)^{-1}$ ,  $\hat{x} = \hat{x}^- + K(y - \hat{x}^-)$ ,  $P = (I - K)P^-$

```
1 def predict(self, acc, dt):
2     # Matrice di transizione cinematica: x=[p,v], input=accelerazione IMU
3     F = np.array([[1.0, dt], [0.0, 1.0]])
4     Bu = np.array([0.5*dt*dt, dt]) # vettore di ingresso (cinematica newtoniana)
5     # Predizione stato: x^- = F x + B u
6     self.x = F @ self.x + Bu * acc
7     # Propagazione covarianza: P^- = F P F^T + Q
8     # Q = q_acc * Bu Bu^T modella il rumore dell'accelerometro
9     self.P = F @ self.P @ F.T + self.q_acc * np.outer(Bu, Bu) + 1e-9 * np.eye(2)
10
11 def correct(self, p_obs, v_obs, noise_scale=1.0):
12     R = self.R_nom * noise_scale # covarianza del rumore di misura
13     S = self.P + R # covarianza dell'innovazione
14     K = self.P @ np.linalg.inv(S) # guadagno ottimale di Kalman
15     self.x = self.x + K @ (np.array([p_obs, v_obs]) - self.x) # correzione
16     self.P = (np.eye(2) - K) @ self.P # aggiornamento covarianza
```

Listing 3.1: estimator.py rr. 26-37: Filtro di Kalman (Eqs. 20-21 di Cui 2022)

Il termine  $q_{acc} \cdot \mathbf{B}_u \mathbf{B}_u^T$  modella un rumore sull'accelerazione (random walk in accelerazione), il che e' fisicamente appropriato per un MEMS IMU che ha un bias lentamente variabile.

### 3.3 File: controller.py (SMC)

#### 3.3.1 Riga 576–587: Pitch Super-Twisting (Algoritmo di Levant)

**Teoria (Super-Twisting):** L'algoritmo Super-Twisting di Levant e' uno SMC del 2° ordine che garantisce convergenza in tempo finito su  $\sigma = 0$  con segnale di controllo **continuo** (elimina il chattering). Data la superficie  $\sigma$ , la legge e':

$$u_1 = -\alpha|\sigma|^{1/2}\text{sgn}(\sigma) + u_2, \quad \dot{u}_2 = -\beta\text{sgn}(\sigma)$$

La funzione di Lyapunov  $V = \alpha^2|\sigma| + \frac{1}{2}u_2^2$  dimostra la convergenza finita.

```

1 def _pitch_super_twisting(self, theta, theta_ref, l):
2     c = self._vlwip(l) # coefficienti VL-WIP alla lunghezza l corrente
3
4     # 1. Superficie di scorrimento: su sigma=0, theta -> theta_ref
5     # esponenzialmente
6     s1 = self.theta_dot + SMC_C1 * (theta - theta_ref)
7
8     # 2. Approssimazione continua del segno (riduce chattering residuo)
9     sg1 = s1 / (abs(s1) + SMC_EPS1)
10
11     # 3. Legge astratta nu (Super-Twisting di Levant):
12     # -C1*theta_dot: termine derivativo (mantiene la superficie)
13     # -KA*sqrt(|s1|)*sgn(s1): termine radice (convergenza finita)
14     # self.W: integratore (aggiornato nel loop: W -= KB*sg1*dt)
15     nu = -SMC_C1*self.theta_dot - SMC_KA*math.sqrt(abs(s1))*sg1 + self.W
16
17     # 4. Feedback Linearization: theta_ddot = a2*theta + b2*(tau_L+tau_R)
18     # Invertendo: tau_comune = (nu - a2*theta) / b2 / 2
19     # (diviso 2: le due ruote erogano la stessa coppia)
20     return 0.5 * (nu - c['a2'] * theta) / c['b2'], s1, sg1

```

Listing 3.2: controller.py SMC rr. 576-587: Pitch Super-Twisting

#### 3.3.2 Righe di Aggiornamento dell'Integratore W nel Loop Principale

```

1 # Nel loop di controllo principale, dopo _pitch_super_twisting:
2 # Integrazione di W_dot = -KB * sgn(s1) (integratore continuo di Levant)
3 # La clip su SMC_W_MAX evita il wind-up dell'integratore
4 self.W = float(np.clip(
5     self.W - SMC_KB * sg1 * min(dt, SMC_DT_MAX),
6     -SMC_W_MAX, SMC_W_MAX))

```

Listing 3.3: controller.py SMC: Aggiornamento dell'integratore di Levant

### 3.3.3 Riga 555–574: Task-Space VMC (Virtual Model Control)

**Teoria (VMC + SMC nel Task Space):** Il VMC di Pratt et al. (1997) comanda forze virtuali cartesiane e le mappa nei giunti:  $\tau = \mathbf{J}^T \mathbf{F}$ . In questa implementazione avanzata,  $\mathbf{F}$  include una superficie di scorrimento cartesiana  $\mathbf{s} = (\mathbf{v}_f - \mathbf{v}_d) + c(\mathbf{p}_f - \mathbf{p}_d)$ :

$$\mathbf{F} = -K_{lin}\mathbf{s} - K_{sw} \text{sat}\left(\frac{\mathbf{s}}{\phi}\right) + \mathbf{F}_{ff}$$

Il termine  $\text{sat}(\mathbf{s}/\phi)$  e' la funzione segno saturata che approssima il controllo discontinuo SMC con un segnale continuo.

```

1 def _leg_smc(self, legs, p_d, c, K_lin, F_ff=np.zeros(2), v_d=np.zeros
  (2)):
2     taus = []
3     for i, ((p_f, J, v_f), qd) in enumerate(zip(legs, self.leg_qd)):
4         # Superficie di scorrimento nel Task Space (errore PD cartesiano
        )
5         # p_f: posizione FK del centro ruota, p_d: setpoint cartesiano
6         s = (v_f - v_d) + c * (p_f - p_d)
7
8         # Forza virtuale composta:
9         # -K_lin @ s: termine proporzionale+derivativo (banda passante)
10        # -LEG_KSW*clip(s/PHI,-1,1): switching saturato (robustezza SMC
        )
11        # F_ff: feedforward peso del robot (compensa la gravita')
12        F = -K_lin @ s - LEG_KSW * np.clip(s / LEG_PHI, -1.0, 1.0) +
        F_ff
13
14        # Feedforward per cancellare lo smorzamento viscoso dell'URDF
15        tau_ff = self.model.p.leg_damping * qd
16
17        # LEGGE VMC: tau = J^T @ F (mappa forze cartesiane in coppie
        giunturali)
18        taus.append(np.clip(J.T @ F + tau_ff, -LEG_TORQUE_MAX,
        LEG_TORQUE_MAX))
19
20        # Riordino per il topic ROS: [hip_L, hip_R, knee_L, knee_R]
21        tau = [taus[0][0], taus[1][0], taus[0][1], taus[1][1]]
22        self.leg_pub.publish(Float64MultiArray(data=[float(x) for x in tau])
        )
23    return tau

```

Listing 3.4: controller.py SMC rr. 555-574: VMC + Sliding Mode nel Task Space

### 3.3.4 Loop Esterno di Velocita' (Non-Minimum Phase)

**Teoria (Sistema a fase non minima):** Il WBR e' un sistema a fase non minima: una coppia positiva alle ruote genera inizialmente uno spostamento del corpo all'indietro (reazione di Newton) prima di accelerare in avanti. Per questo, il loop PI sulla velocita' deve avere guadagni **negativi**  $K_v^P, K_v^I < 0$ : un errore di velocita' positivo (troppo lento) richiede un angolo di pitch negativo (inclinare il robot in avanti) per generare l'accelerazione desiderata.

```

1 def _outer_velocity_loop(self, v, v_ref, dt, active):
2     err = v - v_ref
3     if active:

```

```

4      # Integrale dell'errore di velocita' (con anti-windup)
5      self.Iv = float(np.clip(self.Iv + V_KI * err * dt, -IV_MAX,
6      IV_MAX))
7      # Riferimento di pitch: negativo perche' il sistema e' a fase non
8      minima
9      theta_star = float(np.clip(V_KP * err + self.Iv, -THETA_REF_MAX,
10     THETA_REF_MAX))
11     return theta_star

```

Listing 3.5: controller.py SMC: Loop esterno PI (non-minimum phase)

## Capitolo 4

# rotino\_mpc: Model Predictive Control

### 4.1 Architettura MPC

Il controllore MPC separa il problema in due QP indipendenti seguendo il modello disaccoppiato di Eq. (9) di Cui et al.: uno orizzontale (posizione e velocità nel piano sagittale) e uno verticale (altezza del bacino e forza verticale alle ruote). Questo disaccoppiamento è possibile grazie all'approssimazione del LIP (Linear Inverted Pendulum).

### 4.2 File: solvers.py

#### 4.2.1 Riga 15–30: CARE e Guadagno LQR Ottimale

**Teoria (LQR):** Il controllore LQR minimizza il costo quadratico  $J = \int_0^\infty (x^T Q x + u^T R u) dt$  per il sistema lineare  $\dot{x} = Ax + Bu$ . La soluzione ottimale è  $u^* = -Kx$  con  $K = R^{-1}B^T P$  dove  $P$  soddisfa la CARE:  $A^T P + PA - PBR^{-1}B^T P + Q = 0$ . La soluzione si trova dall'invariante stabile dell'hamiltoniano  $H = \begin{pmatrix} A & -BR^{-1}B^T \\ -Q & -A^T \end{pmatrix}$ .

```
1 def care(A, B, Q, R):
2     n = A.shape[0]
3     Rinv = np.linalg.inv(R)
4     # Costruzione della matrice Hamiltoniana (2n x 2n)
5     H = np.block([[A, -B @ Rinv @ B.T], [-Q, -A.T]])
6     w, V = np.linalg.eig(H)
7     # Autovettori con parte reale negativa (sottospazio stabile)
8     stable = V[:, w.real < 0.0]
9     # Soluzione di Riccati: P = Im_lower * Im_upper^{-1}
10    P = np.real(stable[n:, :] @ np.linalg.inv(stable[:n, :]))
11    return 0.5 * (P + P.T) # simmetrizzazione numerica
12
13 def lqr_gain(A, B, Q, R):
14     P = care(A, B, Q, R)
15     return np.linalg.solve(R, B.T @ P) # K = R^{-1} B^T P
```

Listing 4.1: solvers.py rr. 15-30: CARE e guadagno LQR

#### 4.2.2 Riga 33–45: LQRSchedule (TV-LQR su griglia di lunghezze)

**Teoria (Time-Varying LQR):** Il guadagno LQR ottimale dipende dalla lunghezza  $l$  del pendolo, che varia con la postura delle gambe. Anziché risolvere la CARE a ogni passo

(computazionalmente costoso), si precalcola il guadagno su una griglia di valori di  $l$  e si interpola linearmente a runtime, realizzando un TV-LQR efficiente.

```

1 class LQRSchedule:
2     def __init__(self, model, samples, Q, R):
3         self.l_grid = np.array([s[0] for s in samples])
4         # Precalcolo offline del guadagno ottimale per ogni l nella
         griglia
5         self.K_grid = np.array([lqr_gain(*model.vlwip_matrices(l, I_y=iy
        ), Q, R)
6                                     for l, iy in samples])
7
8     def gain(self, l):
9         l = float(np.clip(l, self.l_grid[0], self.l_grid[-1]))
10        i = int(np.clip(np.searchsorted(self.l_grid, l)-1, 0, len(self.
        l_grid)-2))
11        # Interpolazione lineare del guadagno tra due nodi della griglia
12        a = (1 - self.l_grid[i]) / (self.l_grid[i+1] - self.l_grid[i])
13        return (1.0 - a) * self.K_grid[i] + a * self.K_grid[i+1]

```

Listing 4.2: solvers.py rr. 33-45: LQRSchedule con interpolazione su griglia

#### 4.2.3 Riga 48–58: Fast QP con Algoritmo FISTA

**Teoria (FISTA):** Per QP con vincoli box  $\min_u \frac{1}{2}u^T H u + g^T u$  s.t.  $l_o \leq u \leq h_i$ , il metodo FISTA (Beck & Teboulle 2009) applica discesa del gradiente proiettato con momento di Nesterov:

$$x_{k+1} = \Pi_{[l_o, h_i]} \left( y_k - \frac{H y_k + g}{L} \right), \quad y_{k+1} = x_{k+1} + \frac{t_k - 1}{t_{k+1}} (x_{k+1} - x_k)$$

con  $L = \lambda_{\max}(H)$  e  $t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}$ . La convergenza è  $O(1/k^2)$  anziché  $O(1/k)$  del gradient descent.

```

1 def box_qp(H, g, lo, hi, x0=None, iters=60):
2     # Stima del passo: 1/L dove L = lambda_max(H)
3     L = float(np.linalg.eigvalsh(H)[-1])
4     x = np.clip(np.zeros_like(g) if x0 is None else x0, lo, hi)
5     y, t = x.copy(), 1.0
6     for _ in range(iters):
7         # Passo di discesa proiettato sul box [lo, hi]
8         x_new = np.clip(y - (H @ y + g) / L, lo, hi)
9         # Aggiornamento del momento di Nesterov
10        t_new = 0.5 * (1.0 + np.sqrt(1.0 + 4.0 * t * t))
11        y = x_new + ((t - 1.0) / t_new) * (x_new - x)
12        x, t = x_new, t_new
13    return x

```

Listing 4.3: solvers.py rr. 48-58: FISTA per QP con vincoli box

#### 4.2.4 Riga 61–76: Condensazione del Problema MPC

**Teoria (Condensazione):** Date le dinamiche a spazio di stato  $x_{k+1} = A x_k + B u_k + c$ ,

sostituendo ricorsivamente si ottiene:

$$\mathbf{X} = \Phi x_0 + \Gamma \mathbf{U} + \mathbf{C}_{st}$$

dove  $\mathbf{X} = [x_1; \dots; x_N]$ ,  $\mathbf{U} = [u_0; \dots; u_{N-1}]$ . Le matrici  $\Phi$ ,  $\Gamma$ ,  $\mathbf{C}_{st}$  si costruiscono iterativamente. Il costo quadratico si riscrive come QP puro in  $\mathbf{U}$ :  $H = \Gamma^T S_{blk} \Gamma + W I$ ,  $g = \Gamma^T S_{blk} (\Phi x_0 + C_{st} - X^*)$ .

```

1 def _condense(A, B, c, N):
2     n, m = B.shape
3     Phi = np.zeros((N*n, n)); Gam = np.zeros((N*n, N*m)); Cst = np.zeros
      (N*n)
4     Ak = np.eye(n)
5     for i in range(N):
6         Ak = A @ Ak
7         Phi[i*n:(i+1)*n] = Ak
8         row = i * n
9         prev = Gam[row-n:row] if i > 0 else np.zeros((n, N*m))
10        Gam[row:row+n] = A @ prev
11        Gam[row:row+n, i*m:(i+1)*m] = B
12        Cst[row:row+n] = (A @ Cst[row-n:row] if i > 0 else 0.0) + c
13    return Phi, Gam, Cst
14
15 class UpperBodyMPC:
16     # Disaccoppiamento orizzontale/verticale (Eq. 9 di Cui 2022)
17     def _solve(self, A, B, c, x0, x_ref, S, W, lo, hi, warm):
18         Phi, Gam, Cst = _condense(A, B, c, self.N)
19         Sb = np.kron(np.eye(self.N), S) # matrice di peso a blocchi
20         # Hessiana H e gradiente g del QP condensato
21         H = Gam.T @ Sb @ Gam + W * np.eye(self.N)
22         g = Gam.T @ Sb @ (Phi @ x0 + Cst - x_ref.reshape(-1))
23         warm = np.clip(np.concatenate((warm[1:], warm[-1:])), lo, hi)
24         u = box_qp(H, g, lo, hi, warm)
25         return u, (Phi @ x0 + Gam @ u + Cst).reshape(self.N, -1)
26
27     def solve(self, x_h, ref_h, h, zdd_ref, ds_max, x_v, ref_v, f_min,
      f_max):
28         dt = self.dt
29         # LIP orizzontale: s_ddot = (g+z_ddot)/h * delta_s
30         Bh = np.array([[0.5*dt*dt],[dt]]) * (G + zdd_ref) / max(h, 1e-3)
31         self.u_h, self.x_h = self._solve(self.Ah, Bh, np.zeros(2),
32             x_h, ref_h, self.S_h, self.W_h, -ds_max,
33             ds_max, self.u_h)
34         # Verticale: z_ddot = F_z/m_b - g
35         Bv = np.array([[0.5*dt*dt],[dt]]) / self.m_b
36         cv = np.array([-0.5*dt*dt*G, -dt*G]) # termine costante della
      gravita'
37         self.u_v, self.x_v = self._solve(self.Ah, Bv, cv,
38             x_v, ref_v, self.S_v, self.W_v, f_min,
      f_max, self.u_v)
39         return float(self.u_h[0]), float(self.u_v[0])

```

Listing 4.4: solvers.py rr. 61-114: Condensazione MPC e classe UpperBodyMPC

## Capitolo 5

# rotino\_pid: Controllore Baseline PD

### 5.1 File: controller.py (PID)

#### 5.1.1 Legge PD di Bilanciamento

**Teoria (PD sul pendolo inverso):** La legge PD classica per bilanciare un pendolo inverso su ruote mappa lo stato  $[\theta, \dot{\theta}, \dot{x}, x]$  in coppia alle ruote:  $u = K_{\theta}(\theta - \theta^*) + K_{\dot{\theta}}\dot{\theta} - K_{\dot{x}}(\dot{x} - \dot{x}^*) - K_x(x - x^*)$ . I guadagni sono determinati per tentativi o tramite linearizzazione del sistema (VLWIP) e specifica dei poli desiderati.

```
1 k_th, k_thd, k_xd, k_x, max_wheel_state = balance_gains_for_state(self.  
    jump_state)  
2 wheel_u_raw = clamp(  
3     k_th * (theta - theta_ref)      # P sull'angolo  
4     + k_thd * theta_dot             # D sull'angolo  
5     - k_xd * vel_err                # D sulla velocita' lineare  
6     - k_x * pos_err,               # P sulla posizione  
7     -max_wheel_state, max_wheel_state)
```

Listing 5.1: controller.py PID: Legge PD di bilanciamento (righe 644-648)

#### 5.1.2 Servo PD di Posizione per le Gambe

```
1 def _publish_legs(self, targets, q, qd):  
2     kps = (HIP_SERVO_KP, HIP_SERVO_KP, KNEE_SERVO_KP, KNEE_SERVO_KP)  
3     kvs = (HIP_SERVO_KV, HIP_SERVO_KV, KNEE_SERVO_KV, KNEE_SERVO_KV)  
4     tau = [  
5         clamp(  
6             clamp(kp*(target - q[name]), -LEG_TORQUE_MAX, LEG_TORQUE_MAX)  
7             # P  
8             + clamp(-kv * qd[name], -LEG_DAMPING_MAX, LEG_DAMPING_MAX),  
9             # D (saturato)  
10            -LEG_TORQUE_MAX, LEG_TORQUE_MAX)  
11         for kp, kv, target, name in zip(kps, kvs, targets, LEG_JOINTS)]  
12     self.leg_pub.publish(Float64MultiArray(data=[float(x) for x in tau])  
13 )  
14     return tau
```

Listing 5.2: controller.py PID: Servo PD hip/knee (righe 454-468)

Il termine D e' saturato separatamente ( $\pm \text{LEG\_DAMPING\_MAX}$ ) per evitare cicli limite bang-bang dovuti a picchi di velocita' angolare durante transizioni rapide di postura.

## Capitolo 6

# rotino\_benchmark: Infrastruttura di Validazione

### 6.1 File: campaign.py

```
1 def run_one(law, scenario, duration, out_dir, log_dir):
2     kill_stale()    # rimuove residui da run precedenti
3     launch = ['ros2', 'launch', f'rotino_{law}', f'rotino_{law}.launch.
4     py',
5               'gui:=false', *scenario]
6     logger = ['ros2', 'run', 'rotino_benchmark', 'logger', '--ros-args',
7               '-p', 'use_sim_time:=true', '-p', f'controller:={law}',
8               '-p', f'output_dir:={out_dir}']
9     sim = subprocess.Popen(launch, ...)
10    time.sleep(6.0)    # attesa avvio Gazebo + controllori
11    log = subprocess.Popen(logger, ...)
12    time.sleep(duration) # durata del run
13    # Terminazione ordinata: SIGINT poi SIGKILL
14    for proc in (log, sim):
15        os.killpg(os.getpgid(proc.pid), signal.SIGINT)
16    time.sleep(3.0)
17    for proc in (log, sim):
18        os.killpg(os.getpgid(proc.pid), signal.SIGKILL)
```

Listing 6.1: campaign.py: Ciclo di vita di un run (righe 32-68)

### 6.2 File: logger.py

```
1 def _imu_cb(self, msg):
2     self.imu = msg
3     q = msg.orientation
4     # Estrazione Roll (rotazione attorno all'asse x) dal quaternion
5     sinr_cosp = 2.0 * (q.w * q.x + q.y * q.z)
6     cosr_cosp = 1.0 - 2.0 * (q.x*q.x + q.y*q.y)
7     self.roll = math.atan2(sinr_cosp, cosr_cosp)
8     # Estrazione Yaw (rotazione attorno all'asse z) dal quaternion
9     siny_cosp = 2.0 * (q.w * q.z + q.x * q.y)
10    cosy_cosp = 1.0 - 2.0 * (q.y*q.y + q.z*q.z)
11    self.yaw = math.atan2(siny_cosp, cosy_cosp)
```

Listing 6.2: logger.py: Conversione quaternion in angoli Roll/Yaw

### 6.3 File: plot.py – I 9 Pannelli di Analisi

| N. | File PNG                           | Contenuto e Rilevanza Teorica                                                                              |
|----|------------------------------------|------------------------------------------------------------------------------------------------------------|
| 1  | tracking_and_errors.png            | Errori di tracking di $\theta$ , $x$ , $z$ per confronto risposta transitoria                              |
| 2  | disturbance_and_phase_portrait.png | Profilo di disturbo e ritratto di fase ( $\theta, \dot{\theta}$ ): dimostra la convergenza a 0 dell'orbita |
| 3  | smc_sliding_surface.png            | Superficie $\sigma(t)$ : verifica che il controllore SMC raggiunga e mantenga $\sigma = 0$                 |
| 4  | grf_and_joints.png                 | Ground Reaction Forces e sforzo dei giunti VMC                                                             |
| 5  | torque_distribution.png            | Ripartizione coppia anca/ginocchio e rapporto $ \tau_{hip} / \tau_{knee} $                                 |
| 6  | attitude_3d.png                    | Roll e Yaw vs. tempo: robustezza ai disturbi fuori-piano                                                   |
| 7  | power_and_energy.png               | Potenza meccanica $P = \sum  \tau\omega $ e energia totale (Joule)                                         |
| 8  | horizontal_phase_portrait.png      | Piano delle fasi ( $e_x, \dot{e}_x$ ): dimostra la convergenza all'origine                                 |
| 9  | imu_accelerations.png              | Accelerazioni 3D IMU raw: analisi shock e vibrazioni                                                       |

```

1 def plot_power_and_energy(data, out_dir):
2     for law in ('mpc', 'smc'):
3         d = data[law]
4         power = []
5         energy = [0.0]
6         dt = 0.002 # 500 Hz
7         for i in range(len(d['t'])):
8             # Potenza ruote: P_wheel = |tau_comune * omega_ruota| * 2 (
due ruote)
9             pw = abs(d['u'][i] * d['wheel_v'][i]) * 2.0
10            # Potenza gambe: P_leg = (|tau_hip * omega_hip| + |tau_knee
* omega_knee|) * 2
11            pl = (abs(d['hip_u'][i] * d['hip_v'][i])
12                  + abs(d['knee_u'][i] * d['knee_v'][i])) * 2.0
13            power.append(pw + pl)
14            # Integrazione numerica (rettangoli): E(t) = integral_0^t P
dt
15            energy.append(energy[-1] + (pw + pl) * dt)

```

Listing 6.3: plot.py: Calcolo della potenza meccanica totale istantanea

## Capitolo 7

# Conclusioni

Questo manuale ha dimostrato che ogni riga del codice del progetto RoTino e' la traduzione diretta di un'equazione matematica della teoria del controllo:

- Le **funzioni di rotazione** in `kinematics.py` implementano la geometria del gruppo  $SO(3)$  tramite la formula di Rodrigues e il prodotto di rotazioni RPY.
- Il **modello VL-WIP** in `model.py` estrae automaticamente dall'URDF i parametri del modello lineare di Cui et al. (2022), garantendo coerenza tra simulazione e modello analitico.
- Il **filtro di Kalman** in `estimator.py` implementa le equazioni (20)-(21) del paper, fondendo IMU e odometria in modo ottimale.
- Il **Super-Twisting** in `rotino_smc` garantisce convergenza in tempo finito con segnale continuo (no chattering), tramite l'algoritmo di Levant con integratore aggiornato a ogni ciclo.
- La **condensazione QP** in `solvers.py` trasforma un problema di programmazione dinamica a orizzonte  $N$  in un QP denso di taglia  $N$ , risolvibile in microsecondi con FISTA grazie alla struttura box-constrained.
- Il **benchmark** (`campaign.py`, `logger.py`, `plot.py`) fornisce 9 metriche di validazione che coprono tracking, robustezza ai disturbi, consumo energetico e dinamica di assetto tridimensionale.